

2.2 Writing Functions

Write a function when you repeat a calculation. Test it, check the inputs, and document it.

1. When to write a function

Base R and packages already give you many functions. Write your own when you will do the same work more than once: a conversion, a summary, a plot you rebuild every week.

A function has a name, arguments, a body, and a return value:

```
add_two <- function(a, b) {  
  a + b  
}  
  
add_two(2, 4)  
# [1] 6
```

R returns the last value it evaluates. `return()` is useful when you want to leave early, or when several branches could be the last line. Put `return(...)` on its own line.

A useful order of work:

1. get the logic right on a small example
2. choose a verb-like name
3. wrap the working code in `function(...) { ... }`
4. test ordinary inputs
5. check bad inputs
6. write a short comment or Roxygen header so you remember the contract

2. Duplication is a reason to stop and wrap

This example is the usual one from R for Data Science. Four columns, each rescaled to sit between 0 and 1:

```
set.seed(1)  
df <- tibble::tibble(  
  a = rnorm(10),  
  b = rnorm(10),  
  c = rnorm(10),  
  d = rnorm(10)  
)
```

Copy-paste looks like this:

```
df$a <- (df$a - min(df$a, na.rm = TRUE)) /  
  (max(df$a, na.rm = TRUE) - min(df$a, na.rm = TRUE))
```

Four columns means four copies. It is easy to reuse the wrong column in a `min()` or `max()`, and `min()` is computed twice. `range()` does both ends in one pass:

```
x <- df$a  
rng <- range(x, na.rm = TRUE)  
(x - rng[1]) / (rng[2] - rng[1])
```

That is a function:

```
rescale01 <- function(x) {  
  rng <- range(x, na.rm = TRUE)  
  (x - rng[1]) / (rng[2] - rng[1])  
}  
  
rescale01(c(0, 5, 10))  
# [1] 0.0 0.5 1.0  
  
rescale01(c(1, 2, 3, NA, 5))  
# [1] 0.00 0.25 0.50 NA 1.00
```

Now each column is one line, and a change happens in one place:

```
df$a <- rescale01(df$a)  
df$b <- rescale01(df$b)  
df$c <- rescale01(df$c)  
df$d <- rescale01(df$d)
```

`Inf` is a good test. `range()` treats it as a real endpoint unless you say otherwise:

```
x <- c(1:10, Inf)  
rescale01(x)  
# [1] 0 0 0 0 0 0 0 0 0 0 NaN
```

That is not useful. Ignore non-finite values when you compute the range:

```
rescale01 <- function(x) {
  rng <- range(x, na.rm = TRUE, finite = TRUE)
  (x - rng[1]) / (rng[2] - rng[1])
}

rescale01(x)
# [1] 0.0000000 0.1111111 0.2222222 0.3333333 0.4444444
# [6] 0.5555556 0.6666667 0.7777778 0.8888889 1.0000000      Inf
```

3. if needs one TRUE or FALSE

A conditional runs one block or the other:

```
if (condition) {
  # when condition is TRUE
} else {
  # when condition is FALSE
}
```

The condition must be a **single** logical value. Vectorized `&` and `|` are for elementwise work. Inside `if`, use `&&` and `||`, or collapse a vector with `any()` or `all()`.

Prefer `identical()` to `==` when you want exact equality of two objects. For floating-point comparisons, `dplyr::near()` is safer than `==`.

This function reports which elements of a vector have names:

```
has_name <- function(x) {
  nms <- names(x)
  if (is.null(nms)) {
    rep(FALSE, length(x))
  } else {
    !is.na(nms) & nms != ""
  }
}

has_name(c(1, 2, 3))
# [1] FALSE FALSE FALSE

has_name(c(a = 1, 2, c = 3))
# [1] TRUE FALSE TRUE
```

Chain branches with `else if` when you have several exclusive cases. When one argument chooses among named options, `switch()` is often clearer:

```
do_op <- function(x, y, op) {
  switch(
    op,
    plus = x + y,
    minus = x - y,
    times = x * y,
    divide = x / y,
    stop("Unknown op!")
  )
}

do_op(2, 4, "plus")
# [1] 6

do_op(2, 4, "mod")
# Error in do_op(2, 4, "mod"): Unknown op!
```

Opening `{` stays at the end of the `if` or `function` line. Closing `}` sits on its own line, unless `else` follows it. RStudio's **Code** → **Reindent Lines** will fix indentation.

4. Names, arguments, and scope

Use snake_case: lowercase letters, digits, and `_`. Related functions can share a prefix (`str_`, `fct_`). Do not reuse names of functions you still need, such as `sum` or `length`.

When you call a function, R makes a **new environment** for that call. Arguments live there. A name you define inside the function does not overwrite the same name in the console.

If the function uses a name it did not define, R walks outward until it finds one, or it errors with `object not found`. That is convenient and dangerous. Keep inputs as arguments. Use generic names inside the function (`x`, `y`, `df`) rather than data-specific names from the console.

Put data arguments first, so a pipe can feed them. Give control arguments defaults when there is a sensible usual value:

```
wt_mean <- function(x, w, na.rm = FALSE) {
  stopifnot(is.logical(na.rm), length(na.rm) == 1)
  stopifnot(length(x) == length(w))

  if (na.rm) {
    miss <- is.na(x) | is.na(w)
    x <- x[!miss]
    w <- w[!miss]
  }
  sum(w * x) / sum(w)
}
```

`stopifnot()` checks that each expression is `TRUE`. The error message names the failed check.
`stop("...")` lets you write the message yourself:

```
wt_mean <- function(x, w) {  
  if (length(x) != length(w)) {  
    stop("`x` and `w` must be the same length", call. = FALSE)  
  }  
  sum(w * x) / sum(w)  
}
```

When you call a function, you can leave names off the leading data arguments. Name the later ones (`na.rm = TRUE`) so a new argument in the middle will not silently shift yours.

Look at `?mean`, `?log`, and `?t.test` for the usual pattern: data first, then options with defaults.

5. Comments and Roxygen

A `#` comment should say *why*, not restate the code. A line such as `# load data -----` is a useful section break.

For a function you will keep, write a short header. Roxygen comments start with `#'`. Put the cursor on the `function(` line and use **Code** → **Insert Roxygen Skeleton** (install `roxygen2` once if needed).

```
#' Count positions that are NA in both vectors  
#'  
#' @param x,y Atomic vectors of the same length.  
#'  
#' @return A single integer.  
#'  
#' @examples  
#' both_na(c(1, NA, 3), c(NA, NA, 4))  
both_na <- function(x, y) {  
  stopifnot(length(x) == length(y))  
  sum(is.na(x) & is.na(y))  
}
```

The first line is a short title. `@param` describes each argument. `@return` describes the output. `@examples` should be code that actually runs. `@export` matters when you later turn the file into a package; you can ignore it this week.

6. Practice

1. Recreate `range()` using `min()` and `max()`. Document it.
2. Write `both_na(x, y)`: two vectors of the same length, return how many positions are `NA` in both. Check the inputs. Document it.